

CS6650 Midterm Mastery (Part II)

Sai Karthikeyan Sura

March 2nd, 2026

The experiment extends the Product Search Service with a dependent Inventory microservice. The Product Service enriches search results by calling the Inventory Service for live stock data. The Inventory Service supports chaos engineering via a `/chaos/:mode` API that injects latency, errors, or partial failures on demand.

[Locust :8089] → [Product Service :8080] → [Inventory Service :8081]

Three resilience modes are tested via the `RESILIENCE_MODE` environment variable, each adding progressive protection against cascading failures from the Inventory Service:

Mode	Timeout	Circuit Breaker	Bulkhead	Failure Behavior
none	30s	No	No	Cascading failure
circuit-breaker	500ms	5 fail → open	No	Graceful degradation
bulkhead	500ms	5 fail → open	10 slots	Bounded concurrency

Table 1: Resilience modes and their protection mechanisms.

Phase 1: No Protection

With `RESILIENCE_MODE=none`, the Product Service uses a 30-second HTTP client timeout. When the Inventory Service is slowed to 3 seconds per request via chaos injection, every `/products/search/inventory` call blocks for the full 3 seconds. Each search enriches up to 20 products, so a single search request can block for 60+ seconds.

Baseline (Chaos Off)

With 20 users and the Inventory Service healthy, the system delivered 65.3 RPS with a 3 ms median and 8 ms p95 latency. All requests succeeded with 0% failures.

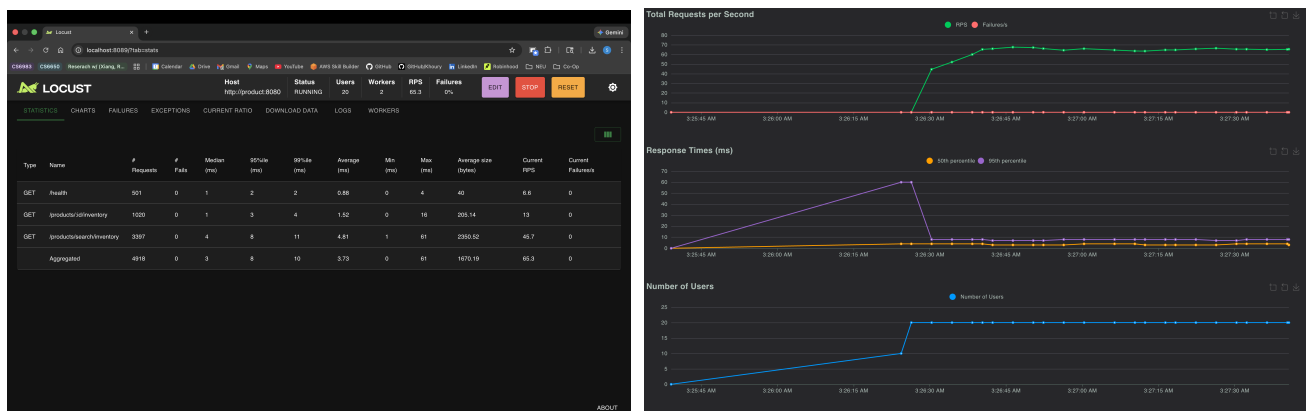


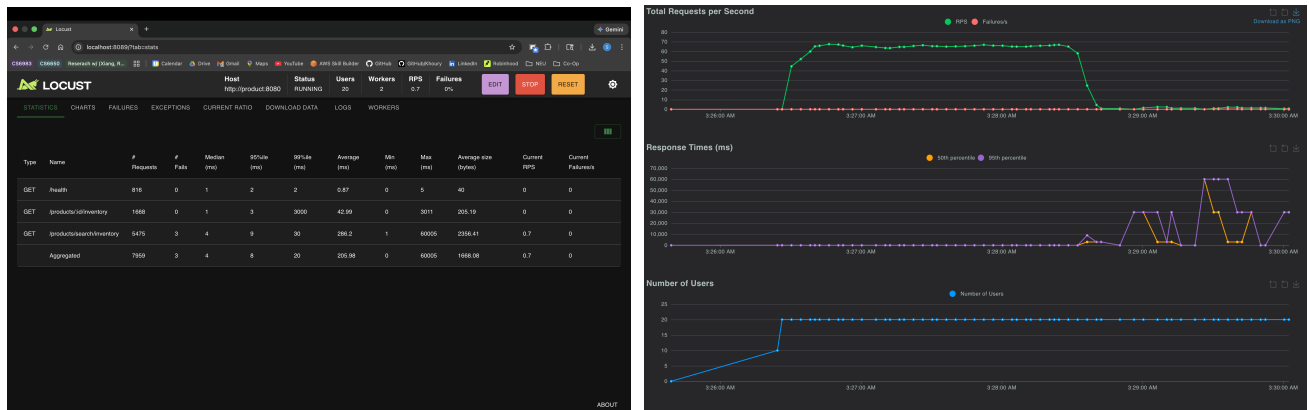
Figure 1: Phase 1 baseline — 65.3 RPS, 3 ms median, 0% failures.

Cascading Failure (Chaos: 3s Delay)

After enabling chaos with a 3-second delay, throughput collapsed from 65.3 RPS to 0.7 RPS — a 99% drop. The p99 latency for `/products/:id/inventory` jumped to 3000 ms (matching the injected delay), while `/products/search/inventory` reached a max of 60,005 ms because each search serially calls inventory for up to 20 products.

Metric	Baseline	Under Chaos
RPS	65.3	0.7
Median	3 ms	4 ms
p95	8 ms	8 ms
p99	10 ms	20 ms
Max	61 ms	60,005 ms
<code>/products/:id/inventory</code> p99	4 ms	3,000 ms

Table 2: Phase 1 performance comparison — baseline vs. cascading failure.



(a) Failure Statistics

(b) Failure Chart

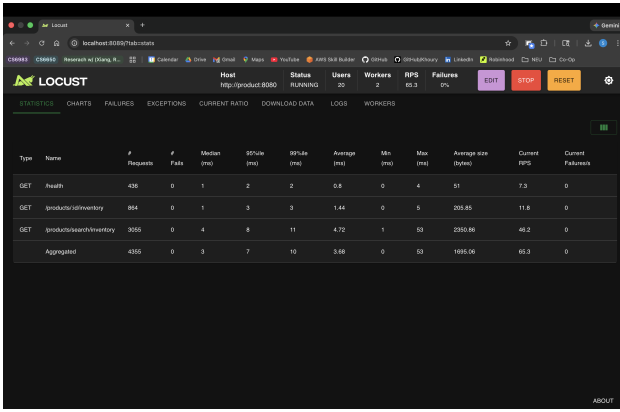
Figure 2: Phase 1 cascading failure — RPS drops to ~ 0 , response times spike to 30–60 seconds.

Phase 2: Circuit Breaker

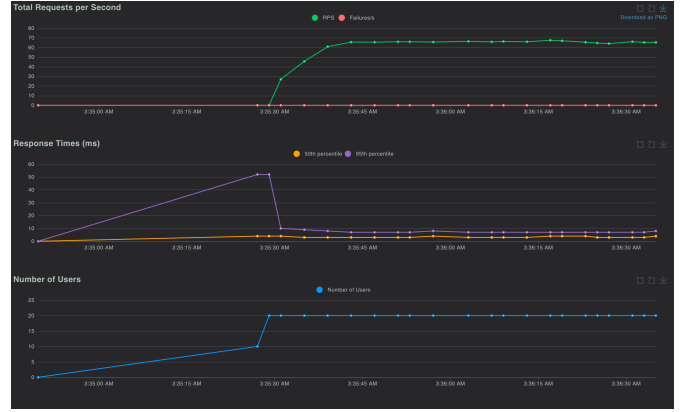
With `RESILIENCE_MODE=circuit-breaker`, two protections activate: a 500 ms fail-fast HTTP timeout (vs. 30s) and a circuit breaker that opens after 5 consecutive failures. When open, all inventory calls return immediately with degraded data (`inventory_source: "circuit-open"`) instead of blocking.

Baseline (Chaos Off)

Baseline performance was identical to Phase 1: 65.3 RPS, 3 ms median, 7 ms p95, 0% failures. The circuit breaker and fail-fast timeout add no measurable overhead when the dependency is healthy.



(a) Baseline Statistics



(b) Baseline Chart

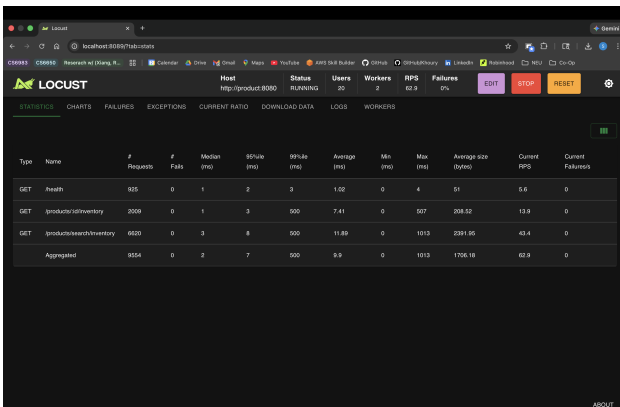
Figure 3: Phase 2 baseline — 65.3 RPS, 3 ms median, and 0% failure rate before fault injection.

Graceful Degradation (Chaos: 3s Delay)

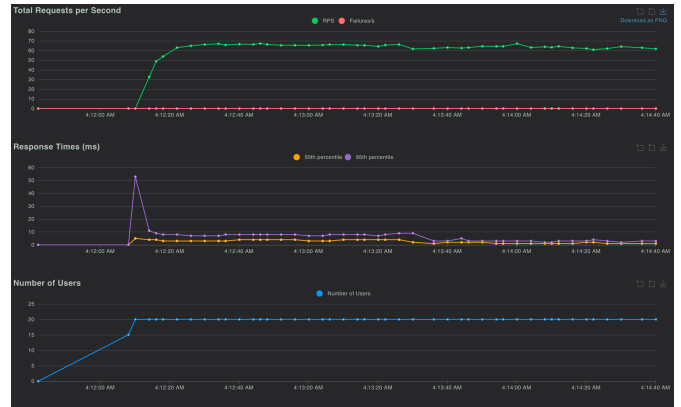
After injecting chaos, the first 5 inventory calls hit the 500 ms timeout, triggering the circuit to open. All subsequent requests returned immediately with degraded data. Throughput stayed at 62.9 RPS (vs. 0.7 RPS in Phase 1). The p99 capped at 500 ms — the fail-fast timeout — compared to 60,005 ms in Phase 1. The median remained at 2 ms because most requests were served from the open circuit path, which skips the inventory call entirely.

Metric	Phase 1 (none)	Phase 2 (CB)
RPS under chaos	0.7	62.9
Median	4 ms	2 ms
p99	20 ms	500 ms
Max	60,005 ms	1,013 ms
Failure rate	0%	0%

Table 3: Phase 1 vs. Phase 2 performance comparison under identical chaos conditions.



(a) Degraded Statistics

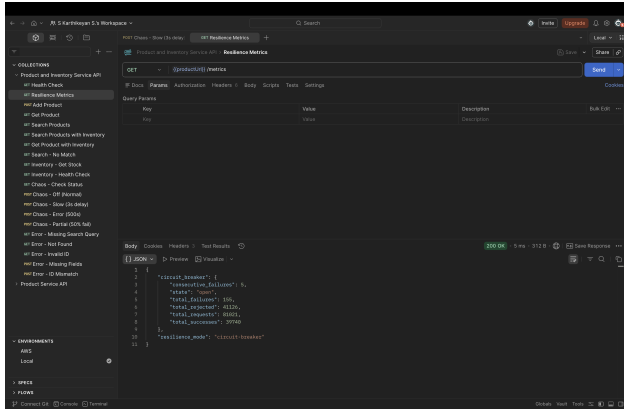


(b) Degraded Chart

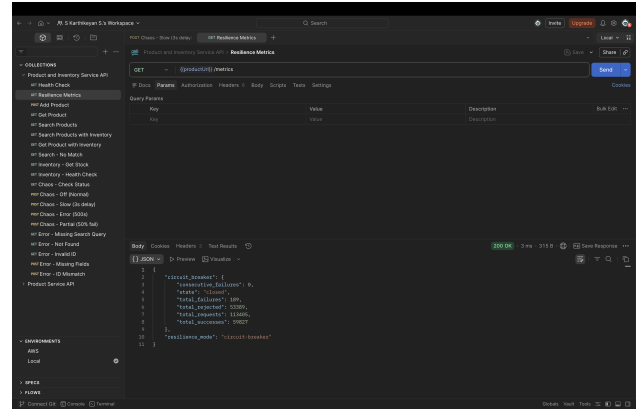
Figure 4: Phase 2 under chaos — 62.9 RPS, 2 ms median, and 0% failure rate after fault injection.

Circuit Breaker Metrics

The `/metrics` endpoint confirmed the circuit breaker transitioned through its state machine: CLOSED → OPEN (after 5 failures) → HALF-OPEN (every 10s to test recovery) → OPEN (test failed while chaos was active) → eventually HALF-OPEN → CLOSED (after chaos was disabled and 3 consecutive successes). During the open state, 41,126 requests were rejected at the circuit breaker level (growing to 53,389 by the time the circuit closed), avoiding tens of thousands of wasteful timeout calls to the broken inventory service.



(a) Open State (Degraded mode)



(b) Closed State (Recovered mode)

Figure 5: Circuit breaker: Fault detection (Open) and recovery (Closed).

State Transition Logs

The service logs in Figure 11 confirm the circuit breaker successfully managed the transition from fault to recovery. After initially moving to the OPEN state due to five consecutive failures, the system entered a cycle of testing recovery via HALF-OPEN transitions. While chaos was active, these tests resulted in a (**test failed**) status, causing the circuit to revert immediately to OPEN. Once the injected latency was removed, a final HALF-OPEN probe succeeded, allowing the system to transition to CLOSED (**recovered**) and resume normal operation.

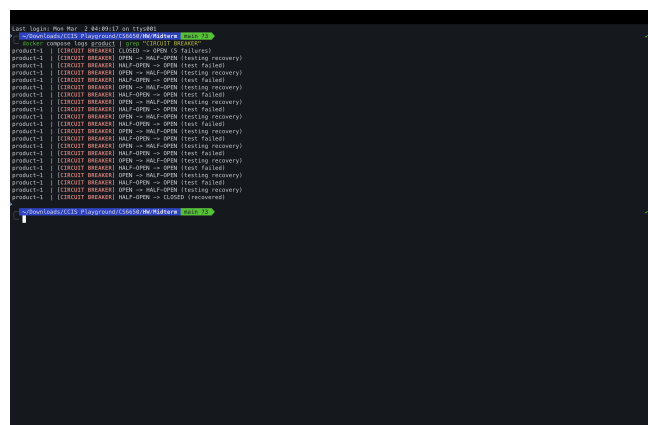


Figure 6: Docker logs showing circuit breaker state transitions.

Live Response After Recovery

Successful system restoration is verified by the manual API response shown in Figure 7. Once the circuit breaker returned to the **CLOSED** state, the **inventory_source** transitioned from fall-

back mode back to "live". The response confirms the Product Service is again successfully communicating with the Inventory microservice, retrieving accurate real-time data.

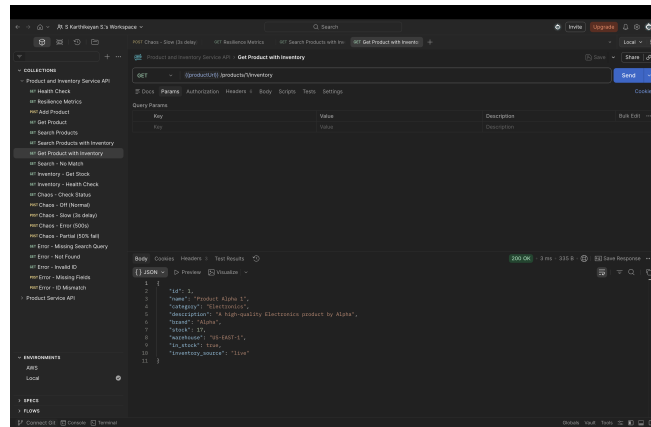


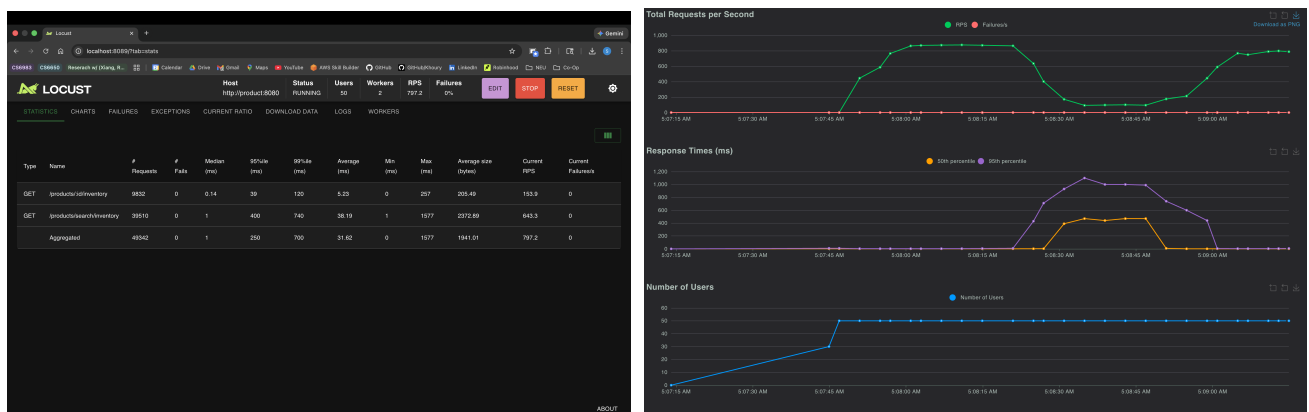
Figure 7: Restored live response following circuit recovery.

Phase 3: Bulkhead

With `RESILIENCE.MODE=bulkhead`, all Phase 2 protections remain active plus a bounded concurrency pool (semaphore with 10 slots) limits the number of in-flight inventory calls. When all 10 slots are occupied, excess requests are immediately rejected with `inventory_source: "bulkhead-rejected"`. This prevents a slow dependency from consuming all server goroutines.

Baseline (Chaos Off, 50 Users)

At 50 users with no chaos, the system delivered 797.2 RPS with 1 ms median and 250 ms p95. The higher user count compared to Phases 1–2 (50 vs. 20) was chosen to demonstrate the bulkhead under heavy concurrency.



(a) Baseline Statistics

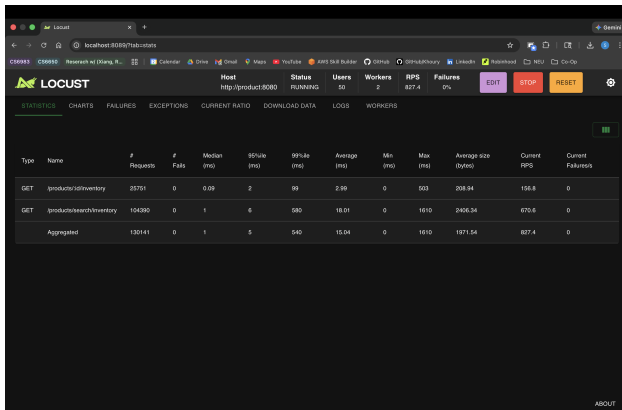
(b) Baseline Chart

Figure 8: Phase 3 baseline — 797.2 RPS at 50 users, 1 ms median.

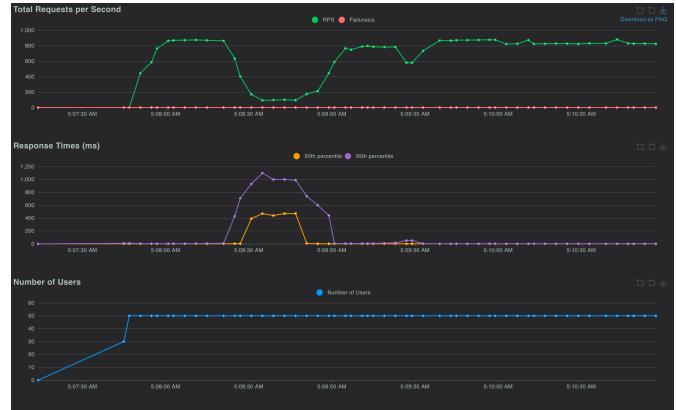
Bounded Degradation (Chaos: 3s Delay)

Under chaos, the bulkhead and circuit breaker worked together. The bulkhead rejected 2,243 requests that exceeded the 10-slot concurrency limit during the brief transition window, while the circuit breaker (which fires first in the code path) rejected 799,347 requests after opening.

Throughput dipped briefly during the transition but recovered to ~ 827 RPS once the circuit fully opened. The system maintained 0% client-visible failures throughout.



(a) Degraded Statistics

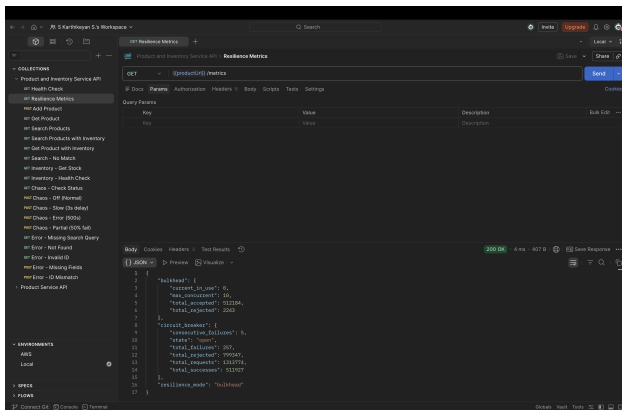


(b) Degraded Chart

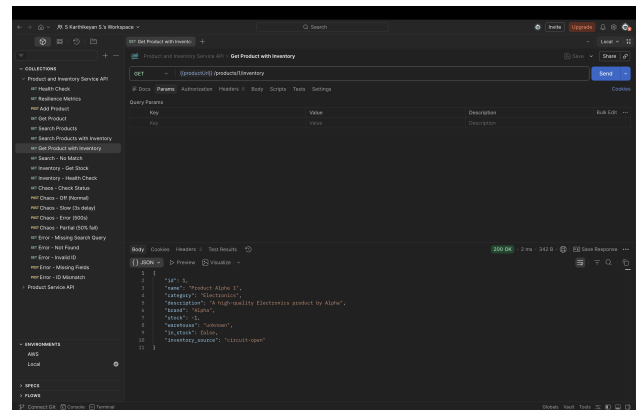
Figure 9: Phase 3 under chaos — circuit breaker transition dip followed by recovery to ~ 800 RPS.

Bulkhead and Circuit Breaker Metrics

The internal system state during high-concurrency chaos is captured in Figure 10. The resilience metrics confirm the **bulkhead** successfully managed saturation by rejecting 2,243 requests that exceeded the 10-slot concurrency limit. Simultaneously, the **circuit_breaker** transitioned to an "open" state after recording 257 total failures, subsequently rejecting 799,347 requests to prevent cascading system load. As shown in the API response, this layered protection ensures that while inventory data is gracefully degraded; returning `inventory_source: "circuit-open"` and a fallback `stock` of -1, the core product information remains available to the user.



(a) Combined Resilience Metrics



(b) Degraded Fallback Response

Figure 10: Phase 3: System metrics and resulting fallback behavior during dependency failure.

State Transition Logs

The service logs in Figure 11 document the automated recovery lifecycle under high-concurrency stress. Initially, the circuit tripped from **CLOSED** to **OPEN** after detecting 5 consecutive failures. During the chaos period, the logs show the system repeatedly entering a **HALF-OPEN** state to probe for recovery; however, these tests failed, forcing an immediate return to the **OPEN** state to maintain system protection. Once the injected latency was removed, a final **HALF-OPEN** test succeeded, allowing the circuit to transition to **CLOSED** (**recovered**) and resume normal operations.

